

Java Streams for Beginners (Tier 2)

A continuation of the Tier 1 guide. These operations show up a lot in interviews, especially when questions involve grouping data, finding items, or doing math.

If you have not gone through the Tier 1 guide yet, please start there first. This guide assumes you already know `filter`, `map`, `collect`, `sorted`, `distinct`, `limit`, `skip`, `count`, and `forEach`.

What's New in Tier 2?

In Tier 1, we learned how to filter items, transform them, and collect them into a list.

In Tier 2, we go further. We learn how to:

- Group items into buckets (like grouping people by city)
- Split items into yes and no piles
- Join strings together with a separator
- Find one item that matches a condition
- Check if any, all, or none of the items match
- Find the smallest or largest item
- Do math on numbers like sum and average
- Combine all items into a single value

Let us go through them one by one.

1. `groupingBy()` : Group Items into Buckets

This is one of the most useful operations in real code and interviews. It groups items based on some key.

The result is always a `Map`. The key is what you grouped by, and the value is a list of items in that group.

Basic example

```
java
```

```

record Person(String name, String city, int age) {}

List<Person> people = List.of(
    new Person("Alice", "Mumbai", 25),
    new Person("Bob", "Delhi", 30),
    new Person("Carol", "Mumbai", 28),
    new Person("Dan", "Delhi", 22),
    new Person("Eve", "Mumbai", 35)
);

Map<String, List<Person>> byCity = people.stream()
    .collect(Collectors.groupingBy(Person::city));

// Result:
// {
//   Mumbai = [Alice 25, Carol 28, Eve 35],
//   Delhi   = [Bob 30, Dan 22]
// }

```

That is it. One line, and you have grouped people by city.

Group strings by length

```

java

List<String> names = List.of("Bob", "Alice", "Dan", "Carol", "Eve");

Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));

// {3 = [Bob, Dan, Eve], 5 = [Alice, Carol]}

```

Group with counting

What if you only want the count in each group, not the full list?

Use `Collectors.counting()` as a second argument.

```

java

Map<String, Long> countByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.counting()
    ));

// {Mumbai = 3, Delhi = 2}

```

This pattern is asked in interviews all the time. Memorize it.

Group with summing

Sum the ages in each city:

```
java

Map<String, Integer> totalAgeByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.summingInt(Person::age)
    ));

// {Mumbai = 88, Delhi = 52}
```

There are also `summingLong` and `summingDouble`.

Group with average

```
java

Map<String, Double> avgAgeByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.averagingInt(Person::age)
    ));

// {Mumbai = 29.33, Delhi = 26.0}
```

Group with mapping (extract a field per group)

What if you only want a list of names per city, not the full Person objects?

Use `Collectors.mapping()` to transform items inside each group.

```
java

Map<String, List<String>> namesByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.mapping(Person::name, Collectors.toList())
    ));

// {Mumbai = [Alice, Carol, Eve], Delhi = [Bob, Dan]}
```

Read it like this: "Group by city. Inside each group, take the names and put them in a list."

Multi-level grouping

You can group inside groups. For example, group by city, then by age range.

```
java
Map<String, Map<String, List<Person>>> nested = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.groupingBy(p -> p.age() < 30 ? "young" : "older")
    ));

// {
//   Mumbai = {young = [Alice 25, Carol 28], older = [Eve 35]},
//   Delhi   = {young = [Dan 22], older = [Bob 30]}
// }
```

This looks complex, but it is just `groupingBy` inside another `groupingBy`.

2. `partitioningBy()` : Split into Yes and No

`partitioningBy` is like `groupingBy`, but only for true and false. It always gives you a Map with two keys: `true` and `false`.

Basic example

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);

Map<Boolean, List<Integer>> evenOdd = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

// {false = [1, 3, 5], true = [2, 4, 6]}
```

Adults vs minors

```
java
Map<Boolean, List<Person>> adultsOrNot = people.stream()
    .collect(Collectors.partitioningBy(p -> p.age() >= 18));

List<Person> adults = adultsOrNot.get(true);
List<Person> minors = adultsOrNot.get(false);
```

`partitioningBy` vs `groupingBy`

You might think, "Why not just use `groupingBy`?"

Two reasons:

1. `partitioningBy` always has both `true` and `false` keys, even if one of them has zero items. `groupingBy` only adds keys for groups that exist.
2. It is slightly faster because it knows there are only two buckets.

If your check is true or false, use `partitioningBy`. If it is a value like a city or category, use `groupingBy`.

Counting in partitions

Like `groupingBy`, you can pass a second collector.

```
java
Map<Boolean, Long> count = people.stream()
    .collect(Collectors.partitioningBy(
        p -> p.age() >= 18,
        Collectors.counting()
    ));

// {false = 0, true = 5}
```

3. joining() : Make a String

`joining` takes a stream of strings and combines them into one string.

Basic example

```
java
List<String> names = List.of("Alice", "Bob", "Carol");

String result = names.stream()
    .collect(Collectors.joining());

// "AliceBobCarol"
```

That is not very useful. Let us add a separator.

With a separator

```
java
```

```
String csv = names.stream()
    .collect(Collectors.joining(", "));

// "Alice, Bob, Carol"
```

With prefix and suffix

```
java

String pretty = names.stream()
    .collect(Collectors.joining(", ", "[", "]"));

// "[Alice, Bob, Carol]"
```

Joining with map

You can use `map` first if your items are not strings.

```
java

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 30)
);

String result = people.stream()
    .map(Person::name)
    .collect(Collectors.joining(", "));

// "Alice, Bob"
```

This is much cleaner than writing a `for` loop with a `StringBuilder`.

4. min() and max() : Find Smallest or Largest

These return the smallest or largest item. They both return an `Optional` because the stream might be empty.

Find min and max numbers

```
java
```

```
List<Integer> nums = List.of(3, 1, 4, 1, 5, 9, 2);

Optional<Integer> min = nums.stream().min(Comparator.naturalOrder());
Optional<Integer> max = nums.stream().max(Comparator.naturalOrder());

System.out.println(min.get()); // 1
System.out.println(max.get()); // 9
```

Find min and max by a field

```
java

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 30),
    new Person("Carol", 28)
);

Optional<Person> youngest = people.stream()
    .min(Comparator.comparingInt(Person::age));

Optional<Person> oldest = people.stream()
    .max(Comparator.comparingInt(Person::age));

System.out.println(youngest.get().name()); // Alice
System.out.println(oldest.get().name());   // Bob
```

Why Optional?

Because the stream could be empty. If you call `min` on an empty list, there is no answer to give. Java forces you to handle this case.

```
java

Optional<Integer> result = List.<Integer>of().stream()
    .min(Comparator.naturalOrder());

if (result.isPresent()) {
    System.out.println(result.get());
} else {
    System.out.println("List is empty");
}
```

Or use `orElse` to give a default:

```
java
```

```
int min = List.<Integer>of().stream()  
    .min(Comparator.naturalOrder())  
    .orElse(-1);
```

5. `findFirst()` and `findAny()` : Get One Item

These both find one item from the stream. Both return an `Optional`.

`findFirst`

Returns the first item in the stream.

```
java
```

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);  
  
Optional<Integer> first = nums.stream()  
    .filter(n -> n > 2)  
    .findFirst();  
  
// Optional[3]
```

`findAny`

Returns any item, often the first one. In a regular stream, it acts the same as `findFirst`.

```
java
```

```
Optional<Integer> any = nums.stream()  
    .filter(n -> n > 2)  
    .findAny();  
  
// Optional[3]
```

When are they different?

In parallel streams, `findAny` can be faster because it does not care about order. It just grabs the first match it can find.

For now, just use `findFirst` when you want the first one. Use `findAny` only if you really do not care which one.

Useful pattern: find first or default

```
java
```



```
Person person = people.stream()
    .filter(p -> p.name().equals("Alice"))
    .findFirst()
    .orElse(null);
```

If Alice is not found, you get `null`. Or you can throw an error:

```
java
Person person = people.stream()
    .filter(p -> p.name().equals("Alice"))
    .findFirst()
    .orElseThrow(() -> new RuntimeException("Not found"));
```

6. anyMatch(), allMatch(), noneMatch() : True or False Checks

These return a `boolean`. They tell you about the items in the stream.

- `anyMatch`: Is at least one item matching?
- `allMatch`: Are all items matching?
- `noneMatch`: Is no item matching?

Examples

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5);

boolean hasEven = nums.stream().anyMatch(n -> n % 2 == 0); // true
boolean allPositive = nums.stream().allMatch(n -> n > 0); // true
boolean noNegatives = nums.stream().noneMatch(n -> n < 0); // true
boolean hasNegative = nums.stream().anyMatch(n -> n < 0); // false
```

With objects

```
java
```

```
record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 17)
);

boolean anyMinor = people.stream().anyMatch(p -> p.age() < 18); // true
boolean allAdults = people.stream().allMatch(p -> p.age() >= 18); // false
```

These are fast

These operations are smart. They stop as soon as they know the answer.

- `anyMatch` stops as soon as it finds one match.
- `allMatch` stops as soon as it finds one that does not match.
- `noneMatch` stops as soon as it finds one match.

So they are very efficient for big lists.

A tricky case: empty stream

What do these return on an empty stream?

```
java

List<Integer> empty = List.of();

empty.stream().anyMatch(n -> n > 0); // false
empty.stream().allMatch(n -> n > 0); // true
empty.stream().noneMatch(n -> n > 0); // true
```

`allMatch` returns `true` on an empty stream. This is because there are no items to break the rule. Interviewers love asking this.

7. `mapToInt()`, `mapToLong()`, `mapToDouble()`: Switch to Primitive Streams

A regular `Stream<Integer>` is a stream of objects. Each number is wrapped in an `Integer` object. This costs memory and time.

For numbers, you can use a special stream that holds primitives directly: `IntStream`, `LongStream`, `DoubleStream`.

Why use these?

1. Faster. No wrapping and unwrapping.

2. They have extra methods like `sum()`, `average()`, and `min()` that work directly without `Optional` wrappers.

Convert with `mapToInt`

```
java

List<String> names = List.of("Alice", "Bob", "Carol");

int totalLetters = names.stream()
    .mapToInt(String::length)
    .sum();
// 13
```

Convert from objects

```
java

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 30),
    new Person("Carol", 28)
);

int totalAge = people.stream()
    .mapToInt(Person::age)
    .sum();
// 83
```

`IntStream.range` and `rangeClosed`

A nice way to make an `IntStream` with a range of numbers.

```
java

// 0 to 4 (5 not included)
IntStream.range(0, 5).forEach(System.out::println);
// 0, 1, 2, 3, 4

// 1 to 5 (5 included)
IntStream.rangeClosed(1, 5).forEach(System.out::println);
// 1, 2, 3, 4, 5
```

This is useful when you need a counter or want to loop a fixed number of times.

8. `sum()`, `average()`, `summaryStatistics()` : Math on Numbers

Once you have an `IntStream`, `LongStream`, or `DoubleStream`, you can do math directly.

`sum()`

```
java  
  
int total = nums.stream().mapToInt(Integer::intValue).sum();
```

`sum` returns a primitive, not an `Optional`. If the stream is empty, you get `0`.

`average()`

```
java  
  
OptionalDouble avg = nums.stream()  
    .mapToInt(Integer::intValue)  
    .average();  
  
System.out.println(avg.getAsDouble());
```

`average` returns an `OptionalDouble` because dividing by zero (empty stream) does not work.

`min()` and `max()` on primitive streams

```
java  
  
OptionalInt min = nums.stream().mapToInt(Integer::intValue).min();  
OptionalInt max = nums.stream().mapToInt(Integer::intValue).max();
```

These return `OptionalInt`, which is like `Optional<Integer>` but for primitives.

`summaryStatistics()`

This one is great. It gives you sum, average, min, max, and count all in one go.

```
java
```

```

IntSummaryStatistics stats = nums.stream()
    .mapToInt(Integer::intValue)
    .summaryStatistics();

System.out.println(stats.getCount());    // 5
System.out.println(stats.getSum());      // 15
System.out.println(stats.getMin());      // 1
System.out.println(stats.getMax());      // 5
System.out.println(stats.getAverage());  // 3.0

```

Real example: stats per group

Combine `groupingBy` with `summarizingInt` to get full stats per group.

```

java

Map<String, IntSummaryStatistics> ageStatsByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.summarizingInt(Person::age)
    ));

IntSummaryStatistics mumbaiStats = ageStatsByCity.get("Mumbai");
System.out.println(mumbaiStats.getAverage());
System.out.println(mumbaiStats.getMax());

```

This is a one-liner that gives you a full report per city. Interviewers love this.

9. reduce() : Combine Everything Into One

`reduce` takes all the items and combines them into a single value. It is more powerful than `sum` or `count`, but also a bit harder to understand.

There are three forms.

Form 1: With a starting value

```

java

List<Integer> nums = List.of(1, 2, 3, 4, 5);

int total = nums.stream()
    .reduce(0, (a, b) -> a + b);

// 15

```

How to read this:

1. Start with `0`.
2. Take the first item (`1`), add it to `0`, get `1`.
3. Take the next item (`2`), add it to `1`, get `3`.
4. Keep going until done. Final answer is `15`.

You can use a method reference too:

```
java
int total = nums.stream().reduce(0, Integer::sum);
```

Multiply all numbers

```
java
int product = nums.stream().reduce(1, (a, b) -> a * b);
// 120
```

Notice we start with `1`, not `0`, because anything times `0` is `0`.

Concatenate strings

```
java
List<String> words = List.of("Hello", " ", "World");

String result = words.stream().reduce("", (a, b) -> a + b);
// "Hello World"
```

Form 2: Without a starting value (returns Optional)

```
java
Optional<Integer> total = nums.stream().reduce((a, b) -> a + b);
total.ifPresent(System.out::println);
// 15
```

It returns `Optional` because the stream might be empty. With no start value and no items, there is nothing to return.

Find max with reduce

```
java
Optional<Integer> max = nums.stream().reduce(Integer::max);
// Optional[5]
```

Yes, you can use `reduce` instead of `max`. But `max` is clearer, so prefer that.

Form 3: With a combiner (advanced)

The third form has three arguments. You will not need this often, but here it is for completeness.

```
java
List<String> words = List.of("Hello", "World");

int totalLength = words.stream()
    .reduce(
        0, // start
        (acc, word) -> acc + word.length(), // add word length to total
        Integer::sum // how to combine results in parallel
    );
// 10
```

The third argument matters only for parallel streams. Skip this for now.

When to use reduce

Honestly, for most cases you can use simpler methods:

- Sum numbers? Use `mapToInt(...).sum()`.
- Count items? Use `count()`.
- Find max? Use `max()`.

Use `reduce` when none of these fit your need, like multiplying numbers or doing custom combinations.

Putting It All Together

Let us look at some real interview-style questions.

Example 1: Group people by city, get the average age per city

```
java
Map<String, Double> avgByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.averagingInt(Person::age)
    ));
```

Example 2: Get a comma-separated list of names of all adults

```
java

String adultNames = people.stream()
    .filter(p -> p.age() >= 18)
    .map(Person::name)
    .collect(Collectors.joining(", "));
```

Example 3: Find the oldest person in each city

```
java

Map<String, Optional<Person>> oldestByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.maxBy(Comparator.comparingInt(Person::age))
    ));

oldestByCity.forEach((city, person) ->
    System.out.println(city + ": " + person.get().name())
);
```

Example 4: Count how many people are in each city

```
java

Map<String, Long> countByCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::city,
        Collectors.counting()
    ));
```

Example 5: Are all people adults?

```
java

boolean allAdults = people.stream().allMatch(p -> p.age() >= 18);
```

Example 6: Find any person over age 30

```
java
```



```
Optional<Person> someone = people.stream()
    .filter(p -> p.age() > 30)
    .findFirst();

someone.ifPresent(p -> System.out.println(p.name()));
```

Example 7: Get total age across all people

```
java

int totalAge = people.stream()
    .mapToInt(Person::age)
    .sum();
```

Example 8: Split people into adults and minors

```
java

Map<Boolean, List<Person>> split = people.stream()
    .collect(Collectors.partitioningBy(p -> p.age() >= 18));

List<Person> adults = split.get(true);
List<Person> minors = split.get(false);
```

Common Mistakes in Tier 2

Mistake 1: Using groupingBy when partitioningBy is enough

```
java

// Works, but not ideal
Map<Boolean, List<Person>> m = people.stream()
    .collect(Collectors.groupingBy(p -> p.age() >= 18));

// Better
Map<Boolean, List<Person>> m = people.stream()
    .collect(Collectors.partitioningBy(p -> p.age() >= 18));
```

`partitioningBy` is cleaner when the key is true or false.

Mistake 2: Forgetting that allMatch returns true on empty streams

```
java
```

```
List<Integer> empty = List.of();
boolean result = empty.stream().allMatch(n -> n > 100);
// true!
```

Always check for empty streams in your test cases.

Mistake 3: Calling get() on an Optional without checking

```
java

Optional<Person> p = people.stream()
    .filter(person -> person.name().equals("Zara"))
    .findFirst();

System.out.println(p.get().name()); // Crash if not found!
```

Use `ifPresent`, `orElse`, or `orElseThrow` instead.

Mistake 4: Using reduce when sum() works

```
java

// Why?
int total = nums.stream().reduce(0, Integer::sum);

// Cleaner
int total = nums.stream().mapToInt(Integer::intValue).sum();
```

Mistake 5: Forgetting about Optional types for primitives

For primitive streams, `Optional` becomes `OptionalInt`, `OptionalLong`, `OptionalDouble`. You call `getAsInt()`, `getAsLong()`, `getAsDouble()` on them, not `get()`.

```
java

OptionalInt maxAge = people.stream()
    .mapToInt(Person::age)
    .max();

int max = maxAge.getAsInt(); // not get()
```

Mistake 6: Using joining without map for non-string streams

```
java
```

```
List<Integer> nums = List.of(1, 2, 3);

// This does NOT compile
String s = nums.stream().collect(Collectors.joining(", "));

// This works
String s = nums.stream()
    .map(String::valueOf)
    .collect(Collectors.joining(", "));
```

`joining` only works on streams of strings.

Quick Cheat Sheet

You want to...	Use
Group items by a key	<code>Collectors.groupingBy(...)</code>
Group and count	<code>Collectors.groupingBy(key, Collectors.counting())</code>
Group and sum	<code>Collectors.groupingBy(key, Collectors.summingInt(...))</code>
Split into yes/no buckets	<code>Collectors.partitioningBy(...)</code>
Join strings	<code>Collectors.joining(", ")</code>
Find smallest	<code>min(Comparator)</code>
Find largest	<code>max(Comparator)</code>
Get first matching item	<code>findFirst()</code>
Get any matching item	<code>findAny()</code>
Check if any match	<code>anyMatch(...)</code>
Check if all match	<code>allMatch(...)</code>
Check if none match	<code>noneMatch(...)</code>
Sum numbers	<code>mapToInt(...).sum()</code>
Average numbers	<code>mapToInt(...).average()</code>
Get all stats at once	<code>mapToInt(...).summaryStatistics()</code>

You want to...	Use
Combine into one value	<code>reduce(...)</code>
Loop with a counter	<code>IntStream.range(0, n)</code> or <code>IntStream.rangeClosed(...)</code>

Final Tips

1. `groupingBy` is the king of Tier 2. Practice it many times. It is asked in almost every Java stream interview.
2. When in doubt about whether to use `Optional`, `OptionalInt`, or just a regular value, look at the return type. The compiler will tell you.
3. Prefer simpler methods (`sum`, `count`, `max`) over `reduce` when they fit. Save `reduce` for cases where nothing else works.
4. Always think about empty streams in your code. What happens if the input list is empty? Most operations handle it nicely, but `Optional` can catch you off guard.
5. The pattern `groupingBy(key, downstream)` covers a huge range of interview questions. Memorize the common downstream collectors: `counting`, `summingInt`, `averagingInt`, `mapping`, `maxBy`, `minBy`.

You now know enough to handle most stream questions in interviews. Practice writing 5 to 10 small stream pipelines a day for a week, and these patterns will feel natural.

When you are ready, move on to Tier 3 for some more advanced concepts like `flatMap`, `peek`, and stream creation.